



EXERCISE — Yakamon

version #1.0.0



Copyright

This document is for internal use at EPITA ([website](#)) only.

Copyright © 2025-2026 Assistants [<assistants@tickets.assistants.epita.fr>](mailto:assistants@tickets.assistants.epita.fr)

The use of this document must abide by the following rules:

- ▷ You downloaded it from the assistants' intranet.*
- ▷ This document is strictly personal and must **not** be passed onto someone else.
- ▷ Non-compliance with these rules can lead to severe sanctions.

Contents

1	Frameworks	8
1.1	Quarkus	8
1.2	Hibernate	8
2	Introduction	8
2.1	Yakamon	8
2.2	Goal	9
2.3	Database	9
3	Architecture	10
3.1	Presentation layer	11
3.1.1	Data Transfer Objects (DTOs)	11
3.1.2	Controllers	11
3.2	Business layer	11
3.2.1	Services	11
3.2.2	Entities	12
3.3	Data layer	12
3.3.1	Repositories	12
3.3.2	Models	12
3.4	Converters	13
4	Testing	14
5	Given annotations	14
5.1	Contexts and dependency injection	14
5.2	Database annotations	15
5.3	Miscellaneous annotations	15
6	How to start	16
6.1	Set up the database	16

*<https://intra.forge.epita.fr>

6.2	Create the database	17
6.3	Given files	17
6.4	Architecture hints	18
7	Useful information	18
7.1	Swagger	18
7.1.1	Swagger document provided	19
7.2	API	19
7.3	Utils folder	19
7.4	Error handling	20
8	Endpoints	20
8.1	Before starting	20
8.2	Context	21
8.3	Game Rules	21
8.4	Start	22
8.5	Player	24
8.6	Move	24
8.7	Catch	24
8.8	Yakadex	25
8.8.1	Yakadex with Query Parameter	26
8.8.2	Specific Yakadex	27
8.9	Team	27
8.10	Rename	27
8.11	Collect	28
8.12	Inventory	28
8.13	Release	29
8.14	Feed	29
8.15	Evolve	30
8.16	Advanced endpoints	30
8.16.1	Move	30
8.16.2	Release	31
9	Environment variables	31

File Tree

```
yakamon/
├── .env
├── .gitignore
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── fr/
│   │   │   │   ├── epita/
│   │   │   │   │   ├── assistants/
│   │   │   │   │   │   ├── yakamon/
│   │   │   │   │   │   │   ├── converter/
│   │   │   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
│   │   │   │   │   │   │   ├── data/
│   │   │   │   │   │   │   │   ├── model/
│   │   │   │   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
│   │   │   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
│   │   │   │   │   │   │   ├── domain/
│   │   │   │   │   │   │   │   ├── entity/
│   │   │   │   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
│   │   │   │   │   │   │   │   ├── service/
│   │   │   │   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
│   │   │   │   │   │   │   ├── presentation/
│   │   │   │   │   │   │   │   ├── api/
│   │   │   │   │   │   │   │   │   ├── *.java (to submit)
```

```

└─ request/
    └─ **/
        └─ *.java (to submit)
└─ response/
    └─ **/
        └─ *.java (to submit)
└─ rest/
    └─ *.java (to submit)
    └─ HelloWorldResource.java
└─ utils/
    └─ **/
        └─ *.java (to submit)
    └─ Direction.java
    └─ ElementType.java
    └─ ErrorCode.java (to submit)
    └─ ErrorInfo.java
    └─ Item.java
    └─ Point.java (to submit)
    └─ tile/
        └─ Collectible.java
        └─ CollectibleInfo.java
        └─ CollectibleType.java
        └─ CollectibleUtils.java
        └─ ItemInfo.java
        └─ ItemType.java
        └─ TerrainType.java
        └─ TileType.java
        └─ YakamonInfo.java
        └─ YakamonType.java
└─ resources/

```

```
├─ application.properties
├─ db/
│   └─ init_db.sql
├─ maps/
│   ├── debug.epimap
│   ├── walkable.epimap
│   └─ yakalos.epimap
├─ openapi.yaml
├─ test/
│   ├── java/
│   │   ├── fr/
│   │   │   ├── epita/
│   │   │   │   ├── assistants/
│   │   │   │   │   ├── yakamon_testsuite/
│   │   │   │   │   │   ├── **/
│   │   │   │   │   │   │   └─ *.java (to submit)
│   │   └── resources/
│   │       ├── **/
│   │       │   └─ * (to submit)
├─ viewer.tar.gz
```

Obligations

Obligations are **fundamental** rules shared by all subjects. They are non-negotiable and to not apply them means to face sanctions. Therefore, do not hesitate to ask for explanations if you do not understand one of these rules.

Obligation #0: Cheating, as well as sharing source code, tests, test tools or coding-style correction tools is **strictly forbidden** and penalized by not being graded, being flagged as a cheater and reported to the academic staff.

Obligation #1: Your submission repository must be **clean**. Except for special cases, which (if any) are **explicitly** mentioned in this document, an *unclean* repository may contain:

- binary files;¹
- files with inappropriate privileges;
- forbidden files: `*~`, `*.swp`, `*.o`, `*.a`, `*.so`, `*.class`, `*.log`, `*.core`, etc.;
- a file tree that does not follow our specifications.

¹If an executable file is required, please provide its sources **only**. We will compile it ourselves.

1 Frameworks

For this project, you will be using two different frameworks: *Quarkus* and *Hibernate*.

1.1 Quarkus

In this project, *Quarkus* will be used to allow you to build a modern backend web application in *Java*. Practically, you will use *Quarkus*' annotations to create REST endpoints.

It was designed for ease of use right from the beginning. It has features that work well with minimal configuration and is considerably less opinionated and more lightweight than other frameworks.

You can use an active record pattern and define your functionalities as entity methods, use a repository pattern, or use services to implement them easily with minimal setup.

Quarkus is optimized for memory density and rapid startup time. Applications running on the Java Virtual Machine (JVM) with *Quarkus* can deliver nearly twice as many instances with the same amount of RAM when compared to other cloud-native *Java* stacks, and up to 7 times more instances when packaged as a native binary.

1.2 Hibernate

In this project, *Hibernate ORM* will be used to handle anything related to the database, directly from your *Java* code.

Hibernate ORM is the de facto standard Jakarta Persistence API (JPA) implementation and provides a fully featured Object-Relational Mapper (ORM). It works beautifully in *Quarkus* and is a powerful tool that allows you to define your entities and relationships in a declarative way.

2 Introduction

2.1 Yakamon

Yakamon is a creature-collecting adventure game where a player explores a grid-based map to collect various resources and capture magical creatures called yakamons. As the player travels through the world, they can gather resources such as scrooges and yakaballs. Captured yakamons are added to the Yakadex, and can be fed and evolved. Your objective is to catch all the yakamons and become the greatest yakamon trainer of all time.

2.2 Goal

The goal of this project is to create the backend of a creature collection game. To do so, you will be asked to implement a **RESTful API**.

You **must** implement multiple REST endpoints to allow the client to communicate with the backend service.

These endpoints are all defined in a Swagger document provided in **YAML** format. You can find it on the intranet.

Your code will be tested in three ways:

- **Architecture tests:** the architecture of your project will be evaluated. Your implementation **should** comply with the *Architecture* section of the subject as they are a good indication of the code review's expectations.
- **API tests:** your backend should return proper responses to requests. These requests and responses are detailed in the Swagger document and explained in the *Endpoints* section. They are all defined in **JSON** format.
- **Database content tests:** the database content will be checked to ensure that all needed operations correctly persisted.

In addition to passing these tests, the project highlights modern software engineering practices like:

- Layered architecture for modularity and maintainability
- API development
- ORM-based data persistence with *Hibernate*

Be careful!

Architecture tests are here to help you identify potential issues and are not part of the grading. They are indicative only and do not replace the code review, which is what will be used for grading. However, you should aim to pass them all, because the code review's expectations focus on respecting the architecture, and failing them would indicate that this is not the case.

2.3 Database

For this project, you **must** use the predefined database described below.

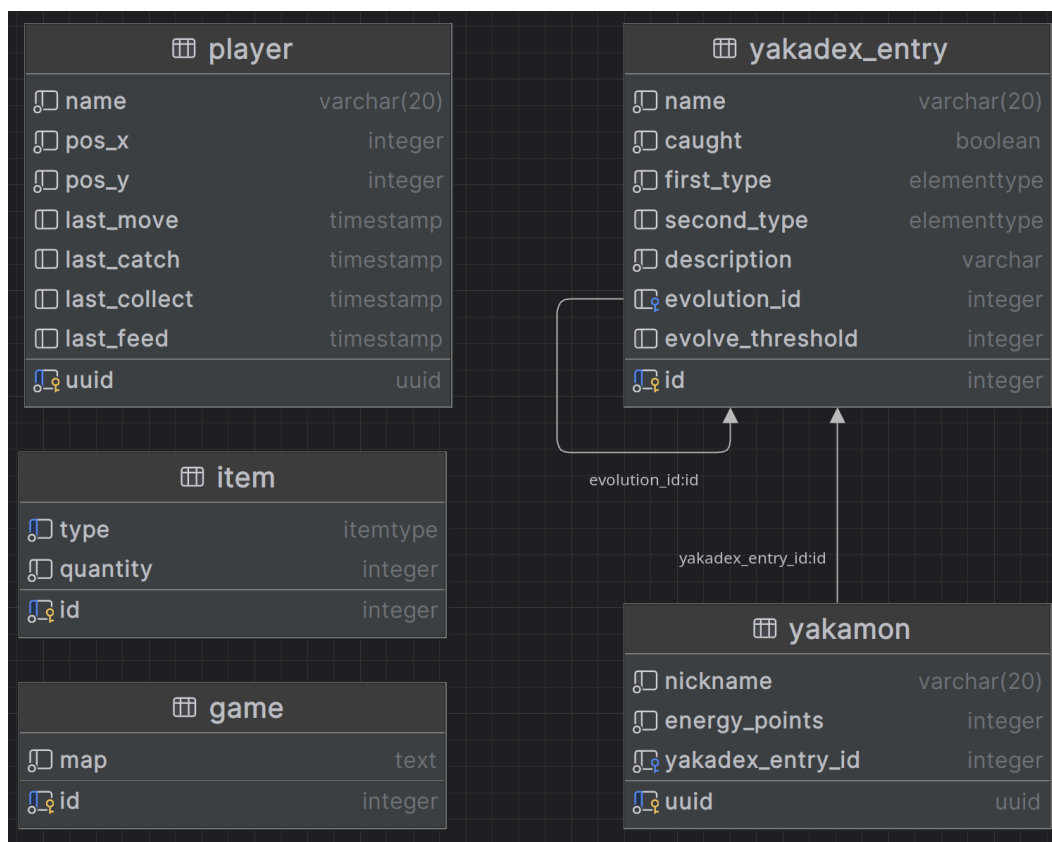


Fig. 1: Database structure for **Yakamon**

3 Architecture

Adhering to the proper architectural pattern is a key objective of this project. Understanding and implementing this pattern will provide a valuable experience for future backend development tasks, whether in professional settings or during internships.

This project follows the **layered architecture** pattern, a widely adopted standard in backend development. While naming conventions may vary between tools and implementations, the core principles remain consistent. Mastering this pattern will enhance your ability to recognize and adapt to similar structures in different environments.

Your project **must** respect the architecture provided in the given files. You can refer to the *architecture hints* section in order to validate the architecture tests.

Tips

You should pay **extreme** attention to the following explanations. It is a difficult concept, so you should read carefully (at least twice). We made a diagram below the explanations to help you visualize the notion, do not hesitate to check it.

3.1 Presentation layer

3.1.1 Data Transfer Objects (DTOs)

DTOs are used to structure and standardize data exchange between Controllers and external clients. They are categorized in two categories:

- **Request DTOs** define the expected structure of client requests.
- **Response DTOs** define the structure of responses sent back to clients.

One of the main reasons DTOs exist is to separate presentation logic from business logic. This allows for business updates without the need to update the whole API for a simple service layer change, as it allows for decoupling. Of course, the API can still change for valid reasons. These changes should be made and communicated properly.

In our case, DTOs are automatically handled by *Quarkus*.

Be careful!

Data Transfer Objects should only be used in the Controller layer. They are contracts that the client must follow in their requests and that the API must adhere to for its responses.

3.1.2 Controllers

Controllers serve as the entry points for external interactions with your backend and are defined in the Presentation layer. Their responsibilities include:

- Defining methods that expose API endpoints via *Quarkus*.
- Validating and handling requests, including error reporting.
- Delegating tasks to Services and returning responses to clients.

Controllers work exclusively with **DTOs** to manage inputs and outputs.

Be careful!

Controllers must **never** interact with the Data layer directly.

3.2 Business layer

3.2.1 Services

Services are the core of your backend. This is where all the logic resides. Each Service is linked to a Repository and its Model. Their responsibilities include:

- Coordinating data retrieval and data updates.
- Implementing the functionalities of the application.

Tips

If you need to work with Model B in Service A, you can call Service B functions in Service A.

Be careful!

Services must **never** interact with DTOs. Instead, you might want to use Entities or primitive types.

3.2.2 Entities

Entities represent the core business objects in the application, typically managed by Services. However, if an Entity is exactly the same as a Model (i.e., it has the same structure and is used for the same purpose), then it is **not** required. In such cases, the Model itself can serve as the data carrier for business logic and should be used directly within Services.

They are only necessary when they provide additional functionality or abstraction beyond the database Model (for example, in aggregation or business-specific logic). If Entities are required, this will be explicitly stated in the project specifications.

3.3 Data layer

3.3.1 Repositories

Repositories manage all database-related operations. Their purpose is to interact with the database, performing tasks such as retrieving and saving data.

Be careful!

Repositories should not contain any business logic. They only aim at interacting with the database and retrieving the needed data. Therefore, you **must** have at most one Model per Repository.

3.3.2 Models

Models represent the database schema and are managed exclusively by Repositories. They provide a structured way to map database records into objects.

Be careful!

Only Repositories and Services can use Models.

3.4 Converters

Converters ensure the independence of each layer in a layered architecture. They are used to transform data objects from one layer's format to another. This helps keep each layer focused on its specific responsibilities. For example, a Converter can:

- Transform a **DTO** from the Presentation layer into an **Entity** for the Service layer.
- Transform an **Entity** from the Service layer into a **Model** for the Data layer.

Using Converters ensures that layers can work together without directly depending on each other's internal data representations. This approach simplifies maintenance and makes it easier to update or extend the application.

Tips

Keep all data transformation logic in Converters to avoid duplicating it in Controllers, Services, or Repositories.

Be careful!

Converters must only handle data transformation. They should not include any business logic or validation.

Here is a simplified diagram to help you visualize the architecture that you **must** respect:

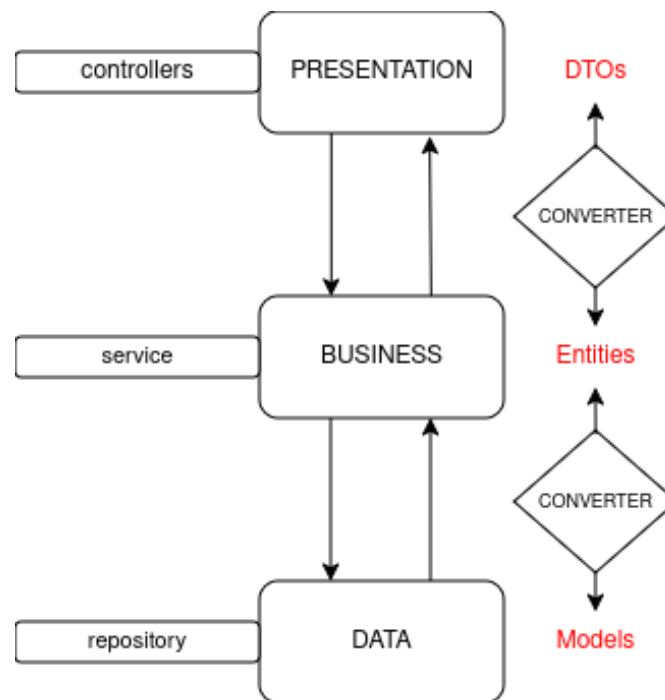


Fig. 2: Layered architecture

4 Testing

As the project is complex and includes multiple components communicating with each other, testing will play a crucial role throughout the project.

You will have to write a complete Testsuite of unit tests using the [JUnit framework](#).

We highly recommend that you read again the *Tests tutorial* from the *Java* workshop.

Each one of your tests must be clearly named, defined, and focused on only one specific aspect of your code. Try to find the corner cases of your functions.

Tips

You may look into the `@QuarkusTest` annotation to test your injected classes and dependencies like converters and services.

As those are functional tests, you do not have to write tests for the API's responses content. However, we highly recommend you do so for the most repetitive cases as it will save you a lot of time.

Going further...

You may look into this [RestAssured tutorial](#) to learn more about functional testing of Rest APIs.

5 Given annotations

5.1 Contexts and dependency injection

The instantiation of classes in order to use them can be repetitive. To simplify code, *Quarkus* provides built-in annotations to manage contexts and inject dependencies.

Imagine that we have a class `Logger`, which allows us to log messages. We may want to inject it in other classes to use it. Then we can write this code:

```
@ApplicationScoped
class Logger {
    public void log(String message) {
        // Logging function
    }
}

class MyService {
    @Inject
    Logger logger;

    public void doSomething() {
        logger.log("Hello World!");
    }
}
```

Here, the annotation `@ApplicationScoped` specifies that we want to be able to instantiate a `Logger` whenever we want in any class.

The annotation `@Inject` allows us to get this instance. The default behavior of *Quarkus* is to generate only one instance of the class, and then give that instance to every class that needs it.

Tips

These annotations could be very useful to communicate between layers.

5.2 Database annotations

- `@Transactional`: this annotation is used to explicitly state that a method is adding or removing elements in the database.

```
@Transactional
public void resetDatabase() {
    database.deleteAll();
}
```

Be careful!

The `@Transactional` annotation ensures that all changes made within a single method are executed as a single transaction. However, this can cause errors when deleting and then adding elements to a database within the same transaction.

- `@Enumerated`: this annotation specifies that an enum attribute in a model should be converted to a different type when stored in the database. In the shown example, the `@Enumerated` annotation automatically converts the `EnumExample` enum into a `String` representing its key when stored in the database.

```
@Entity
@Table(name = "my_table")
class MyTable {
    @Enumerated(EnumType.STRING)
    public EnumExample example;
}
```

Be careful!

The `@Enumerated` annotation is required in *Yakamon*; otherwise, the enum type will default to `smallint` in the database.

5.3 Miscellaneous annotations

- `@Value`: this annotation indicates that our class is a value object. It automatically generates its getters and constructors, as well as other useful methods.
- `@With`: this annotation generates methods such as `withX` for each attribute of the class. These methods can be chained together to create a new instance of the class, initialized with the values provided in the *with* calls.

```
@With
class FooBar() {
    public String foo;
    public String bar;
}

class Main() {
    public static void main(String[] args) {
        new FooBar().withFoo("foo").withBar("bar");
    }
}
```

- `@ConfigProperty`: this annotation allows us to retrieve the value of an environment variable.

```
import org.eclipse.microprofile.config.inject.ConfigProperty;

@ConfigProperty(name="foobar") String foobar;
```

Be careful!

Make sure you are using the `@ConfigProperty` from the `microprofile` library. Other `@ConfigProperty` annotations exist but do not behave the same way.

6 How to start

6.1 Set up the database

First, let us configure some *PostgreSQL*-specific environment variables:

```
42sh$ echo 'export PGDATA="$HOME/postgres_data"' >> ~/.bashrc
42sh$ echo 'export PGHOST="/tmp"' >> ~/.bashrc
42sh$ echo 'export PGPORT="5432"' >> ~/.bashrc
42sh$ source ~/.bashrc
```

This first line adds a `PGDATA` environment variable to your `.bashrc`, containing the location of your choice to store *PostgreSQL*'s data. The second specifies the host name of the machine on which the server will run. As the value begins with a slash, it will be used as a directory containing the socket on which *Postgres* will listen. The third one sets the port on which the server will listen. If you ever need to modify these environment variables, do not forget to reload your shell configuration.

Then, let us initialize a new *PostgreSQL* database cluster. It will generate default databases and configurations in the `$PGDATA` directory.

```
42sh$ nix-shell -p postgresql
42sh$ initdb --locale "$LANG" -E UTF8
```

Be careful!

If you experience a permission denied error, you may need to restore your rights using `chmod 755 ~`.

6.2 Create the database

You can now start the *postgres* server by running:

```
42sh$ postgres -k "$PGHOST"
```

By default, on the PIE, you do not have a *postgres* superuser. You can create one by running the following command:

```
42sh$ createuser -s postgres
```

Now that the *postgres* server is running, you can create the database by running the following commands:

```
42sh$ createdb -U postgres yakamon
```

Be careful!

Make sure to repeat these steps every time you restart your computer.

6.3 Given files

To begin the project smoothly, you *should* open the given files with IntelliJ before reading the upcoming sections of the subject.

Once your project is opened in your IDE, you can type the following commands to start the server and the viewer:

```
42sh$ mvn clean install
42sh$ mvn quarkus:dev
```

It will launch your API on <http://localhost:8081>. We have provided you with a Hello World endpoint, so the next command will return "Hello World!"

```
42sh$ curl http://localhost:8081/hello
```

To help you test and debug your code, we provide a viewer. The viewer can be used to ease your debugging process in a fun way but does not replace the moulinette.

To use the viewer, navigate to the `viewer` folder and run the `./start.sh` command. To stop it you can just use `Ctrl+C` where you have run the command. If the port is already in use you can find and kill the process using it with the command `lsof -i :3010`.

Once launched, you can access it at the following URI:

- <http://localhost:3010>

6.4 Architecture hints

In order to validate our architecture tests, you **must** respect the following architecture:

- **converter**: this directory **must** contain all the logic related to the converters. Each Converter **must** end with Converter like TestConverter.
- **data**: this directory **must** contain all the logic related to the Model layer.
 - **model**: this directory **must** contain all your database models. Each Model class **must** end with Model like TestModel.

Be careful!

All your models **must** have getters and setters.

- **repository**: this directory **must** contain all your models' repositories. Each Repository class **must** end with Repository like TestRepository.
- **domain**: this directory **must** contain all the logic related to the Service layer.
 - **entity**: this directory **must** contain all your entities. Each Entity class **must** end with Entity like TestEntity.
 - **service**: this directory **must** contain all your services. Each Service class **must** end with Service like TestService.
- **presentation**: this directory **must** contain all the logic related to your controllers.
 - **api**: this directory **must** contain all your Request and Response DTOs.
 - ★ **request**: this directory **must** contain all your requests. Each Request **must** end with Request like TestRequest.
 - ★ **response**: this directory **must** contain all your responses. Each Response **must** end with Response like TestResponse.
 - **rest**: this directory **must** contain all your rest controllers. Each REST class **must** end with Resource like TestResource.

7 Useful information

7.1 Swagger

Swagger is a powerful tool used to define, design, and document REST APIs. It provides a standardized way to describe your API endpoints, as well as parameters, responses, and expected behaviors. Swagger allows you to:

- Clearly define your API structure, including endpoints, and request and response formats.
- Automatically generate API documentation.
- Test your APIs easily using tools like Swagger UI.

7.1.1 Swagger document provided

For this project, you are provided with a Swagger document, defining the behaviors of Yakamon.

The Swagger document can be found at: `yakamon/src/main/resources/openapi.yaml`.

This file describes the REST API endpoints for interacting with the game.

Tips

Use the provided Swagger documents to guide the implementation of your API and ensure compliance with the defined specifications. Swagger UI can be used to visualize and test the endpoints for easier debugging and validation. You can use IntelliJ or a website such as <https://editor-next.swagger.io/> to do so.

Tips

Once you have completed an API endpoint, you can test the behavior of your project without using `curl`. Instead, to test it directly on the web, you can add `/q/swagger-ui` in your web URL, like so: <http://localhost:8081/q/swagger-ui>.

Be careful!

The swagger will help you clearly visualize the expected behavior for the different endpoints. We **strongly** advise you to use it.

7.2 API

An API (Application Programming Interface) is a way for programs to communicate with each other. In this project, you will create a backend that provides endpoints for the frontend to call.

The **frontend** represents the part of the program that users interact with. It will send requests to your backend using this API to update the behavior of the program. For example, when the frontend wants to collect resources, it will call an API endpoint in your backend to perform the action and get a response.

Your job is to ensure the backend processes these requests correctly and answers with the required data, or a meaningful error message if something goes wrong.

7.3 Utils folder

The `utils` folder provides already defined classes and utilities that may be used in multiple layers throughout the project.

It is designed to centralize and standardize shared logic and data structures. This ensures consistency and avoids duplication.

You will find a folder `tile` which contains useful classes to represent tiles in the game and other files such as `Point` to represent coordinates. You can add other utility classes if needed, such as a `Logger` class for logging purposes.

Be careful!

You must submit the `Point.java` file if you use it. However, modifying its content is not required to complete the project. The possibility is here if you want to update it with additional useful methods.

7.4 Error handling

Whenever an error occurs during the execution of an endpoint, you **must** return an appropriate HTTP status code and an error message.

The response body **must** follow the *JSON* format below:

```
{
  "message": "content"
}
```

Tips

You can return a custom class in order to get the proper format.

Throwing errors can be repetitive and heavy. To address this issue, we provide the utils class `ErrorInfo`, as well as the enum `ErrorCode` that is up to you to complete. Their use is not mandatory but **strongly** advised.

8 Endpoints

8.1 Before starting

All the details regarding HTTP methods, paths, parameters, and responses for the following endpoints can be found in the Swagger document. We strongly advise you to keep this document open at all times. You will need to use environment variables. To test your code's behavior, you can write them in the provided `.env` files.

Be careful!

You **must not** allow duplicates in the database. Either create a new entry if it does not exist, or update the existing one.

Be careful!

For all endpoints except `Start`, you must ensure that the game has already started.

Be careful!

If a field value is null you must not include this field in the response. A Jackson annotation might help you for this.

Be careful!

Some last things you need to know before starting:

- When working with models, you **must** use *Java*'s standard *camelCase* convention for variable names.
- Dates **must** be stored as `LocalDateTime` to ensure precise timestamps, and the item type **must** be stored as `ItemType` rather than as a `String`.
- Primitive types **must** be avoided; use their boxed counterparts instead.

8.2 Context

Welcome to the world of **Yakamons**!

A world full of wonders and mysteries inhabited by magical creatures, yakamons.

Across the lands of **Yakalos**, people and creatures live side by side. Some forge lifelong friendships, others test their might in battle, and a rare few seek to gather every yakamon ever discovered.

As a young yakamon trainer, you will explore a large world, face many challenges, and of course, catch as many yakamons as possible to complete the **Yakadex**, the one and only yakamon encyclopedia.

You landed on a vast map filled with diverse biomes, each harboring unique yakamons and valuable resources. Your goal is to **explore** these lands, **collect** resources known as scrooges and yakaballs, **catch** wild yakamons and **evolve** them to become the greatest yakamon trainer of all time!

8.3 Game Rules

Before starting, here are some rules to help you better understand how the game works.

There is only one player in the game. The player starts with **5 yakaballs** in their inventory.

Tips

The inventory corresponds to the items stored in the item database.

The player has a **team** of yakamons. When a new one is caught, it is added to the team of the player.

The player also has a **Yakadex**, an encyclopedia featuring information on every yakamon available in the game.

A **Yakadex** entry features various information about a yakamon; those are its id and its name. When a yakamon is caught for the first time, the player gets to learn more about it as it unlocks more information such as a description and its types.

Be careful!

A player can only have **3 yakamons** in their team at a time! But do not worry, they **can release** their creatures. A released yakamon is **still registered** as caught in the **Yakadex**.

Be aware! Some yakamons *cannot be found in the wild* and must be evolved from other yakamons.

Now that you are aware of the game rules, you can start your journey! To begin, introduce yourself with the `start` endpoint.

Be careful!

Unless explicitly specified, any request body with a missing or blank field is considered **invalid** and **must** be answered with an `Error`.

8.4 Start

This endpoint starts a new game.

Before proceeding, you **must** reset the game state by setting all yakamons as uncaught in the `yakadex` and by clearing the other tables.

Then, you **must** :

- Set your trainer name, and your `x` and `y` coordinates to 0.
- Add 5 `Yakaballs` into your inventory.

Since you will later need to check whether you recently moved, collected resources, caught yakamons or fed them, you **must** also set the corresponding attributes to `null`.

Be careful!

Do not forget to update these attributes later in the following endpoints.

Finally, you **must** read and parse the map at the given path.

The map file is encoded in a similar manner to the **RLE format**, except it contains two letters after each digit, one for the terrain and one for the collectible (scrooge or yakamon) present on the tile.

Tips

If the tile does not contain any resource, we will use 'N' to specify it.

The terrain type should have the following names:

- G: Grass
- W: Water
- S: Sand
- M: Mountain
- R: Rock
- L: Lava

The collectibles should have the following names:

- S: Scrooge
- Y: Yakaball
- N: No collectible
- y: Yakimon

- n: Nicotali
- o: Louicune
- g: Grouvan
- b: Bastiedon
- l: Lucarya
- r: Rayquaïssa
- k: Koyako

There are many others yakamons, we let you explore the given files to discover them all!

Tips

Resources always use uppercase letters and yakamons lowercase letters.

As an example, a file in our version of the RLE format would be:

```
42sh$ cat special_rle_format.txt
3GN1GS
1RY2WN1Rr
```

Here, we have four grass tiles, one with a scrooge on it; one rock tile with a yakaball on it; two water tiles without any collectibles; and one rock tile with a wild yakamon named *Rayquaïssa*. This should be stored as 3GN1GS;1RY2WN1Rr in the database.

You **must** return the map object created after parsing the file.

You can find maps in the *resources* folder and use their **absolute path** to test the behavior of your code.

Be careful!

You **should** work with entities to handle the game, as working with a grid of tiles is more efficient than using a *String*.

Tips

Do not forget to persist everything you created in the database. With *Hibernate*, you can use a simple method to do so.

Tips

Each block count in this format (e.g., 3GN) must be a single digit, from 1 to 9.

Tips

You do not have to handle files that are not correctly formatted.

8.5 Player

This endpoint retrieves your data stored in the *player* database.

8.6 Move

Great, your journey is about to begin, but you will not go far if you cannot move.

This endpoint makes you move in a specific direction. Make sure to check that the tile is walkable and that the direction is valid.

Tips

You might consider looking at the given enum `TerrainType`.

You should use the *Environment variables* below to determine the timing for movement:

- `JWS_TICK_DURATION`
- `JWS_MOVEMENT_DELAY`

The `JWS_TICK_DURATION` *environment variable* represents a number of milliseconds and is used as a unit of time. The `JWS_MOVEMENT_DELAY` *environment variable* represents the number of ticks to wait before being able to move again.

You **must** return your new position on success.

Be careful!

For error handling, you **must** first check that the game is running and that you have not moved too recently before anything else.

8.7 Catch

Now that you can move, it is time to catch your first yakamon. To do so, you need to move to its exact position, then throw a `Yakaball` with the `catch` endpoint. You can catch multiple yakamons of the same species.

This endpoint generates a new `Yakamon` and stores it into the database.

To be able to catch a yakamon, you must satisfy all the following rules:

- You **must not have** caught a yakamon too recently.
- You **must be** at the same coordinates as a wild yakamon.
- Your inventory **must contain** at least one yakaball.
- You **must have** less than 3 yakamons in your team.

You should use the *Environment variables* below to determine the catch delay:

- `JWS_TICK_DURATION`
- `JWS_CATCH_DELAY`

If those conditions are all met, you must:

- Remove one yakaball from your inventory.
- Create a new yakamon with a random UUID and the information of the yakamon you are catching.
- Add this new yakamon to your team.
- Set the yakamon's species as caught in the Yakadex.
- Return the newly created Yakamon.

Tips

An annotation might help you to let Quarkus generate this value automatically.

The species of your newly caught Yakamon **must** be the same as the one you found, and its *nickname* **must** have the wild yakamon's name as a default value. The energy level of a newly caught yakamon is 0.

Tips

The species of a yakamon can be retrieved using the *yakadexId* of the yakamon in the *yakadex* database.

Be careful!

You **must** not forget to remove the wild yakamon from the map once it has been caught. To do so, you can set the tile's second character to 'N'.

Do not forget to persist the changes to the database.

8.8 Yakadex

Congratulations on your first capture! This is the first of many. But what did you catch exactly? Let us check using the Yakadex!

The yakadex endpoint gives you information about every available yakamon.

Be careful!

For every uncaught yakamon, only its *name*, its *id* and its *caught* status can be shown in the response.

It is also a way to check which yakamons you have already caught.

Here is a response example we could get when calling this endpoint:

```
{
  "entries": [
    {
      "id": 1,
      "name": "Yakimon",
      "firstType": "FIRE",
      "evolveThreshold": 5,
      "evolutionId": 2,
```

(continues on next page)

```

    "caught": true,
    "description": "Each and every time it inhales and exhales, hot embers dribble out of ↵
↳ its mouth and nostrils"
  },
  {
    "id": 2,
    "name": "Yakoryu",
    "firstType": "FIRE",
    "secondType": "FIGHTING",
    "caught": true,
    "description": "It is not afraid of combat, its fiery punches are feared by many"
  },
  {
    "id": 3,
    "name": "Rayquaïssa",
    "caught": false
  }
]
}

```

Tips

As Yakoryu does not have an evolution, the fields `evolveThreshold` and `evolutionId` are not in the response.

8.8.1 Yakadex with Query Parameter

This endpoint supports the **query parameter** `only_missing`, when set to `true`, it filters out already caught yakamons.

This parameter is **optional** and its default value is `false`.

Hence, when set, we should get this:

```

{
  "entries": [
    {
      "id": 3,
      "name": "Rayquaïssa",
      "caught": false
    }
  ]
}

```

8.8.2 Specific Yakadex

Furthermore, you can get information about a specific yakamon using the **path parameter** id.

Here is an example of what should return `/yakadex/2` :

```
{
  "id": 2,
  "name": "Yakoryu",
  "firstType": "FIRE",
  "secondType": "FIGHTING",
  "caught": true,
  "description": "It is not afraid of combat, its fiery punches are feared by many"
}
```

8.9 Team

While the Yakadex gives you information about all yakamons in the world, you might want to check which yakamons are currently in your team.

Remember that as a trainer, you can only have a **maximum** of 3 yakamons in your team at the same time.

This endpoint retrieves the yakamons that are currently in your team. These are the yakamons you can feed and show off to your friends proudly!

8.10 Rename

Now that you can see your team of yakamons, you might want to give them personal nicknames. After all, each yakamon is unique, and giving them a special name strengthens the bond between a yakamon and its trainer.

This endpoint lets you change one of your yakamon's nickname. It takes its id from path parameters (e.g., `/team/:uuid/rename`) and its new nickname in the request body.

Be careful!

Do not mistake yakamon uuids with yakadex ids, those are for the species.

You must fulfill all of these conditions in order to rename a yakamon:

- The yakamon with the given *uuid* **exists** in your team.
- The new nickname **is not** blank or null and cannot exceed 20 characters.

If those conditions are met, you **must** set its new nickname, persist the changes, and return the updated yakamon.

8.11 Collect

After a long day of exploring and catching yakamons, you might find yourself low on yakaballs. In this case, you can use the `collect` endpoint to gather resources from the tile you are standing on.

This endpoint lets you collect resources from the tile you are currently standing on.

The available resources are:

- S for Scrooge object
- Y for Yakaball object

To be able to collect a resource:

- You **must be standing** on a tile that has a resource.
- You **must not** have collected too recently.

You should use the *Environment variables* below to determine the collect delay and how many resources to add to the inventory:

- JWS_TICK_DURATION
- JWS_COLLECT_DELAY
- JWS_COLLECT_MULTIPLIER

If all the conditions to collect are met, you **must**:

- Add the resource to your inventory.
- Remove it from the map tile.
- Persist the changes to the database.

8.12 Inventory

Now that you have some resources, you might want to check what is in your inventory.

This endpoint gives you information about your inventory.

Tips

As we allow no duplicates in the database, each resource type should appear only once; the quantity field indicates how many of that resource you own.

8.13 Release

You discovered the yakamon that you have dreamt about ever since you were a kid, but your team is full. Fortunately, you can `release` one of your current yakamons to make space for a new one.

This endpoint lets you release one of your yakamons. It takes its *uuid* from path parameters.

To be able to release a yakamon, you need to have a yakamon with its *uuid* present in the database. If that is the case, you **must** delete it and then persist the changes in the database.

Tips

You do not have to handle the 403 error code for the moment.

8.14 Feed

One of your yakamons is staring at you intently. It looks like it is hungry. Why would you not feed it with a little scrooge and help it get stronger?

This endpoint lets you feed one of your yakamons to increase its energy points. For every scrooge eaten, a yakamon earns one energy point.

To be able to feed a yakamon, you need to fulfill these conditions in this exact order:

- You **must not** have fed a yakamon too recently.
- The yakamon with the corresponding *uuid* **exists** in your team.
- You **must have** the required resources in your inventory.

You should use the *Environment variables* below to determine the feed delay:

- JWS_TICK_DURATION
- JWS_FEED_DELAY

If those conditions are met, you **must**:

- Decrease the resources by the used quantity in your inventory.
- Increase the yakamon's energy points accordingly.
- Persist the changes to the database.
- Return the updated yakamon information in the response.

Be careful!

You must not delete a resource entry when its quantity reaches 0.

Tips

A Yakamon does not have an energy point limit. Even if the `evolve_threshold` is reached, or it does not have an evolution, it can be fed.

8.15 Evolve

Scrooges increase the energy points of yakamons. When a certain threshold is reached, yakamons can use those points to **evolve** into a different form.

Would you help your yakamon reach its full potential by evolving it?

This endpoint allows you to evolve a yakamon to its next form.

To be able to evolve your yakamon, you need to fulfill these conditions:

- The yakamon with the corresponding *uuid* **exists** in your team.
- The yakamon **must** have an evolution stage.
- The yakamon **must** have at least the amount of energy points required to evolve.

Tips

A Yakamon's evolution stage and its required energy points to evolve can be found using the `evolution_id` and `evolution_threshold` fields of its corresponding `YakadexEntry` in the database.

If all conditions are met, you **must**:

- Change its species value to the evolved form specified in its yakadex entry.
- Set the evolved form's `caught` as `true` in the yakadex.
- If its nickname is the same as its species' name, change it to the evolved species' name.
- Decrease its energy points by its evolution cost.
- Persist it in the database.
- Return the updated yakamon information in the response.

8.16 Advanced endpoints

Now that you have a functional game, you might want to add some advanced features to it.

8.16.1 Move

You are now allowed to walk on unwalkable terrains if you have a yakamon of a specific type in your team. The yakamon's secondary type also counts.

Tips

This feature should be added in the already existing `move` endpoint.

To be able to move on unwalkable terrains, you **must** have a yakamon in your team with the required type to walk on that terrain.

The required types for each terrain are:

- `WATER` type allows you to walk on Water tiles.

- FIRE type allows you to walk on Lava tiles.
- ROCK type allows you to walk on Mountain tiles.
- FLYING type allows you to walk on all kinds of tiles.

You should return your new position on success like you did before.

8.16.2 Release

As you are now able to move on new tiles thanks to your partners' abilities, you may run into serious problems. What happens if you decide to release your last yakamon able to swim while moving on water? As the yakamons care for their trainer, they would never let them put themselves in danger.

To prevent yourself from making a fatal mistake, you will need to do some updates to the `release` endpoint to handle these cases.

Tips

If you have two or more yakamons able to swim, the release of one of them should not raise any error.

9 Environment variables

Name	Type	Example
JWS_TICK_DURATION	Integer	100
JWS_MOVEMENT_DELAY	Integer	2
JWS_CATCH_DELAY	Integer	5
JWS_COLLECT_DELAY	Integer	3
JWS_FEED_DELAY	Integer	4
JWS_COLLECT_MULTIPLIER	Integer	2

Table 1: Yakamon environment variables

The `JWS_TICK_DURATION` *environment variable* represents a number of milliseconds and is used as a unit of time. Thus, you should consider every delay as an amount of ticks.

Knowledge is a paradox... The more one understands, the more one realizes the vastness of his ignorance.